Workshop 5: Concatenating and merging

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn
NHH Norwegian School of Economics

October 30, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/TECH2-H26

Contents

1 Exercise 1: Business cycle correlations 1

2 Exercise 2: Loading many data files 2

3 Exercise 3: Weekly returns of the Magnificent Seven 2

4 Exercise 4: Inflation expectations and past inflation 3

Exercise 1: Business cycle correlations

For this exercise, you'll be using macroeconomic data from the folder `data/FRED`.

The monthly files contain the following variables:

Variable	Description
`DATE`	Month of observation.
`CPI`	Consumer Price Index (1982–1984 = 100).
`UNRATE`	Unemployment rate, in percent.
`FEDFUNDS`	Effective federal funds rate, in percent.
`REALRATE`	One-year real interest rate, in percent.
`LFPART`	Labor force participation rate, in percent.

Part 1 — Loading and concatenating data

1. There are seven decade-specific files named `FRED_monthly_YYYY.csv` where `YYYY` identifies the decade by its first year (`YYYY` takes on the values 1950, 1960, . . . , 2010). Write a loop that reads in all seven files as DataFrames and stores them in a list.

   Hint: Recall from the lecture that you should use `pd.read_csv(..., parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates.

2. Use `pd.concat()` to concatenate these data sets into a single `DataFrame` and set the `DATE` column as the index.

Part 2 — Merging GDP

You realize that your data set does not include GDP since this variable is only reported at a quarterly frequency. The file contains real GDP, which adjusts for changes in prices. Load the GDP data from the file `GDP.csv` and merge it with your monthly data using an *inner join*.

Explain what happens to the number of observations when you merge the monthly and quarterly data sets.

Part 3 — GDP growth and unemployment

GDP measures how much an economy produces, and `UNRATE` is the unemployment rate. We want to explore how changes in these two variables are related.

1. Create a *new* `DataFrame` containing percent changes in GDP and percentage-point changes in `UNRATE`.
   - Compute GDP growth with `pct_change()` and multiply by 100 to convert to a percentage.
   - Compute changes in unemployment with `diff()`. For example, an increase in unemployment from 6% to 7% is a change of 1 percentage point.
2. Use `corr()` to compute the correlation between GDP growth and changes in unemployment. What do the sign and magnitude of the correlation coefficient tell you?
3. Create a scatter plot with GDP growth on the horizontal axis and the change in unemployment on the vertical axis. Each point should represent one quarter. Add horizontal and vertical lines at zero. Where are the quarters in which GDP falls and unemployment rises? Does unemployment always rise when GDP falls?

Exercise 2: Loading many data files

Python can find files matching a pattern using `glob.glob()`.

Load the seven decade files `FRED_monthly_YYYY.csv` in the local `data/FRED` folder using `glob.glob()`. Match names starting with `FRED_monthly_1` or `FRED_monthly_2`, and store the DataFrames in a list. Parse `DATE` as dates and set it as the index. Concatenate the list along the row axis and sort by date.

Exercise 3: Weekly returns of the Magnificent Seven

In this exercise, you are asked to analyze the weekly stock market returns of the so-called Magnificent Seven, which are some of the most successful tech companies of the last few decades: Apple (AAPL), Amazon (AMZN), Alphabet/Google (GOOGL), Meta (META), Microsoft (MSFT), Nvidia (NVDA), and Tesla (TSLA).

The data for this exercise is located in the folder `data/stockmarket/`. Each stock file contains the following variables:

Variable	Description
`Date`	Trading date.
`Open`	Opening stock price for the trading day.
`Close`	Closing stock price for the trading day.

Part 1 — Loading and computing returns

For each of the seven stocks listed above, there is a corresponding CSV file in this directory based on the ticker symbol. For example, the file for Apple is `AAPL.csv`.

1. For each ticker symbol, load the corresponding CSV file and make sure that the `Date` column is set as the index.

   The DataFrame has two columns, `Open` and `Close`, which contain the opening and closing prices for each trading day.

2. Use `resample()` to resample the daily data to a weekly frequency by specifying `resample('W')`, and compute the weekly returns in percent:

$$\text{Weekly returns} = \frac{\text{Close price on last day} - \text{Open price on first day}}{\text{Open price on first day}} \times 100$$

   Hint: You can obtain the first and last observations using the `first()` and `last()` methods.

3. Append these returns to a list so you can merge them into a single DataFrame later.

Part 2 — Merging weekly returns

Merge the weekly returns into a single DataFrame, keeping the intersection of dates available for all 7 stocks. Discard the first and last common weeks, which are incomplete in these files.

Hint: This can be achieved using either `pd.concat(..., join='inner')`, `pd.merge()`, or `DataFrame.join()`.

Part 3 — Pairwise correlations

Finally, you are interested in how the weekly returns are correlated across the 7 stocks.

1. Compute and report the pairwise correlations using `DataFrame.corr()`.
2. Create a figure with a 7-by-7 grid of subplots showing the pairwise scatter plots of weekly returns for each combination of stocks.

   You can do this either with the `scatter_matrix()` function contained in `pandas.plotting`, or manually build the figure using Matplotlib functions.

3. **[Advanced]** Add the corresponding correlation coefficient to each off-diagonal subplot (e.g., about 0.43 for AAPL and AMZN).

Exercise 4: Inflation expectations and past inflation

In this exercise, we explore whether people's expectations about future inflation are related to inflation they have recently experienced. Inflation measures how much prices rise, while deflation means that prices fall.

We use a 10% sample of Survey of Consumer Expectations (SCE), a survey of US consumers administered by the Federal Reserve Bank of New York. The files in `data/SCE_10pct/` contain individual responses from June 2013 through September 2025. Missing values and extreme forecasts have already been removed. Each row represents one respondent in one survey month.

Variable	Description
`userid`	Respondent identifier.
`date`	First day of the survey month.

Variable	Description
`female`	Gender indicator: 0 = male, 1 = female.
`infl_1y`	Expected inflation over the next 12 months, in percent.

For example, `infl_1y = 3.5` means that the respondent expects prices to rise by 3.5% over the next 12 months. A negative value indicates expected deflation.

Part 1 — Loading and grouping

1. Write a loop to load the 13 annual files `SCE_2013.csv`, ..., `SCE_2025.csv` into a list of DataFrames. Parse `date` as a date column.

   Hint: Use `pd.read_csv(..., parse_dates=['date'])` to automatically parse strings stored in the `date` column as dates.

2. Concatenate these DataFrames into a single DataFrame using `pd.concat()` and sort the rows by `date` and `userid`.

3. Group by `date` and `female` to calculate mean expected inflation for each month and gender. Keep the result as a DataFrame, rename `infl_1y` to `expected`, and use `reset_index()` to make the grouping variables regular columns again.

   Hint: Use `rename()` to rename columns.

Part 2 — Past inflation and merging

The Consumer Price Index (CPI) measures the overall level of consumer prices. For example, if the CPI rises from 100 to 104 over 12 months, inflation over that period is 4%.

1. Load `data/FRED/CPI.csv`, parse `DATE` as dates, and rename it to `date`.

2. Compute inflation over the preceding 12 months and store it in a new column named `past_inflation`:

$$\text{Past inflation}_t = \left(\frac{CPI_t}{CPI_{t-12}} - 1 \right) \times 100.$$

   Here, t is the current month and $t - 12$ is the same month one year earlier.

   Hint: Use `pct_change(periods=12)` and multiply by 100. Compute this change on the monthly CPI data before merging.

3. Use `pd.merge()` to merge `past_inflation` with the monthly expectations from Part 1, matching on the `date` column. Keep both genders for every survey month. Why do men and women receive the same observed inflation rate in each month?

Part 3 — Plotting and interpretation

1. Create a figure with two scatter plots, one for men and one for women. Put observed inflation over the *past* 12 months on the horizontal axis and mean expected inflation over the *next* 12 months on the vertical axis. Each point should represent one month.

   Add axis labels and panel titles. Use the same axis limits in both panels so that we can compare the groups.

2. Are higher past inflation rates associated with higher inflation expectations? Does the relationship differ by gender?